

# E-Commerce Sales SQL Analysis Portfolio

Brazilian E-Commerce Dataset • SQLite

17 production-quality queries across 8 analysis domains

JOINS • Aggregations • CTEs • Window Functions • BI Reporting

■ JOINS

■ GROUP BY

■ Aggregations

■ Revenue  
Analysis

■ Window  
Functions

■ CTEs

■ BI Reporting

## SECTION 1: DATABASE OVERVIEW KPIS

### KPI 1: Total Customers

```
SELECT COUNT(*) AS Total_Customers
FROM customers;
```

### KPI 2: Total Orders

```
SELECT COUNT(*) AS Total_Orders
FROM orders;
```

### KPI 3: Total Products

```
SELECT COUNT(*) AS Total_Products
FROM products;
```

### KPI 4: Total Sellers

```
SELECT COUNT(*) AS Total_Sellers
FROM sellers;
```

## SECTION 2: ORDER ANALYSIS

### Query 5: Order Status Analysis

```
SELECT
order_status,
COUNT(*) AS total_orders
FROM orders
GROUP BY order_status
ORDER BY total_orders DESC;
```

### Query 6: Monthly Orders Trend

```
SELECT
  strftime('%Y-%m', order_purchase_timestamp) AS Month,
  COUNT(*) AS Total_Orders
FROM orders
GROUP BY Month
ORDER BY Month;
```

## SECTION 3: PRODUCT ANALYSIS

### Query 7: Top Product Categories

```
SELECT
  c.product_category_name_english AS Category,
  COUNT(*) AS Total_Items_Sold
FROM order_items oi
JOIN products p
  ON oi.product_id = p.product_id
JOIN categories c
  ON p.product_category_name = c.product_category_name
GROUP BY Category
ORDER BY Total_Items_Sold DESC
LIMIT 10;
```

### Query 8: Top Product Categories by Revenue

```
SELECT
  c.product_category_name_english AS Category,
  ROUND(SUM(pay.payment_value),2) AS Revenue
FROM orders o
JOIN payments pay
  ON o.order_id = pay.order_id
JOIN order_items oi
  ON o.order_id = oi.order_id
JOIN products p
  ON oi.product_id = p.product_id
JOIN categories c
  ON p.product_category_name = c.product_category_name
GROUP BY Category
ORDER BY Revenue DESC
LIMIT 10;
```

## SECTION 4: PAYMENT ANALYSIS

### Query 9: Payment Method Analysis

```
SELECT
  payment_type,
  COUNT(*) AS Total_Transactions
FROM payments
GROUP BY payment_type
ORDER BY Total_Transactions DESC;
```

### Query 10: Average Order Value

```
SELECT
ROUND(AVG(payment_value),2) AS Average_Order_Value
FROM payments;
```

## SECTION 5: REVENUE ANALYSIS

### Query 11: Monthly Revenue Trend

```
SELECT
strftime('%Y-%m', o.order_purchase_timestamp) AS Month,
ROUND(SUM(p.payment_value),2) AS Revenue
FROM orders o
JOIN payments p
ON o.order_id = p.order_id
GROUP BY Month
ORDER BY Month;
```

### Query 12: Monthly Revenue Growth Percentage

```
WITH MonthlyRevenue AS (
SELECT
strftime('%Y-%m', o.order_purchase_timestamp) AS Month,
ROUND(SUM(p.payment_value),2) AS Revenue
FROM orders o
JOIN payments p
ON o.order_id = p.order_id
GROUP BY Month
)

SELECT
Month,
Revenue,
ROUND(
((Revenue - LAG(Revenue) OVER (ORDER BY Month))
/ LAG(Revenue) OVER (ORDER BY Month)) * 100,
2
) AS Growth_Percentage
FROM MonthlyRevenue;
```

## SECTION 6: CUSTOMER ANALYSIS

### Query 13: Top Customer Cities by Revenue

```
SELECT
c.customer_city,
c.customer_state,
ROUND(SUM(p.payment_value),2) AS Revenue,
COUNT(DISTINCT o.order_id) AS Total_Orders
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
JOIN payments p
ON o.order_id = p.order_id
GROUP BY
c.customer_city,
c.customer_state
ORDER BY Revenue DESC
LIMIT 10;
```

### Query 14: Top Customer Cities by Number of Orders

```
SELECT
c.customer_city,
c.customer_state,
COUNT(o.order_id) AS Total_Orders
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY
c.customer_city,
c.customer_state
ORDER BY Total_Orders DESC
LIMIT 10;
```

---

## SECTION 7: SELLER ANALYSIS

### Query 15: Top Sellers

```
SELECT
s.seller_id,
s.seller_city,
s.seller_state,
COUNT(*) AS Total_Items_Sold
FROM order_items oi
JOIN sellers s
ON oi.seller_id = s.seller_id
GROUP BY
s.seller_id,
s.seller_city,
s.seller_state
ORDER BY Total_Items_Sold DESC
LIMIT 10;
```

### Query 16: Top Seller Cities by Revenue

```
SELECT
s.seller_city,
s.seller_state,
ROUND(SUM(p.payment_value),2) AS Revenue
FROM sellers s
JOIN order_items oi
ON s.seller_id = oi.seller_id
JOIN orders o
ON oi.order_id = o.order_id
JOIN payments p
ON o.order_id = p.order_id
GROUP BY
s.seller_city,
s.seller_state
ORDER BY Revenue DESC
LIMIT 10;
```

## SECTION 8: ADVANCED SQL — WINDOW FUNCTIONS

### Query 17: Rank Product Categories by Revenue

```
SELECT
c.product_category_name_english AS Category,
ROUND(SUM(p.payment_value),2) AS Revenue,
RANK() OVER (
ORDER BY SUM(p.payment_value) DESC
) AS Revenue_Rank
FROM orders o
JOIN payments p
ON o.order_id = p.order_id
JOIN order_items oi
ON o.order_id = oi.order_id
JOIN products pr
ON oi.product_id = pr.product_id
JOIN categories c
ON pr.product_category_name = c.product_category_name
GROUP BY Category;
```

## Project Summary

This portfolio contains **17 SQL queries** demonstrating real-world data analysis skills on a Brazilian e-commerce dataset. Techniques covered include multi-table JOINS, aggregation functions, date formatting, CTEs, and advanced window functions (RANK, LAG) suitable for a Data Analyst role.